Question:

Develop a **"Library Management System"** to demonstrate Object-Oriented Programming concepts.

Execution Steps:

STEP 1:

Open IntelliJ IDEA.

STEP 2:

Create a Java project.

STEP 3:

Create a package named:

javacore

STEP 4:

Create a Java class named:

LibMangSys

STEP 5:

Write all the given classes and interfaces in the program.

STEP 6:

Save the program.

STEP 7:

Compile the program.

STEP 8:

Run the program.

STEP 9:

Execution starts from the main() method of LibMangSys.

STEP 10:

The program displays:

Book Details

STEP 11:

A Book object b1 is created using the default constructor.

The default values are:

Book Id = 123

Book Name = Malgudi Days

Author = RkNarayan

Price = 299

STEP 12:

A second Book object b2 is created using the parameterized constructor.

The values are:

Book Id = 108

Book Name = Life

Author = Thomas

Price = 500

STEP 13:

displayBook() displays the details of b1 and b2.

STEP 14:

The program displays:

Inheritance

STEP 15:

A Student object is created with:

Name = Ram

Age = 20

Roll No = 18

STEP 16:

The Student constructor calls the Person constructor using super().

STEP 17:

displayStudent() displays the student details.

STEP 18:

A Faculty object is created with:

Name = Ramesh

Age = 38

Subject = Maths

STEP 19:

The Faculty constructor calls the Person constructor using super().

STEP 20:

displayFaculty() displays the faculty details.

STEP 21:

The program displays:

Overloading

STEP 22:

An Area object is created.

STEP 23:

calculateArea(4) calculates the area of a square.

Result = 16

STEP 24:

calculateArea(5, 6) calculates the area of a rectangle.

Result = 30

STEP 25:

calculateArea(9.0) calculates the area of a circle.

Result = 254.34

STEP 26:

The program displays:

Overriding

STEP 27:

Vehicle reference v1 refers to a Car object.

STEP 28:

Vehicle reference v2 refers to a Bike object.

STEP 29:

v1.display() calls the overridden display() method of Car.

Output:

This is a car

STEP 30:

v2.display() calls the overridden display() method of Bike.

Output:

This is a bike

STEP 31:

The program displays:

Abstraction

STEP 32:

A Circle object and Rectangle object are created using Shape references.

STEP 33:

c.draw() calls the draw() method of Circle.

Output:

Draw circle

STEP 34:

r.draw() calls the draw() method of Rectangle.

Output:

Draw rectangle

STEP 35:

The program displays:

Interface

STEP 36:

A Report object is created.

STEP 37:

Report implements the Printable interface.

STEP 38:

re.print() calls the print() method of Report.

STEP 39:

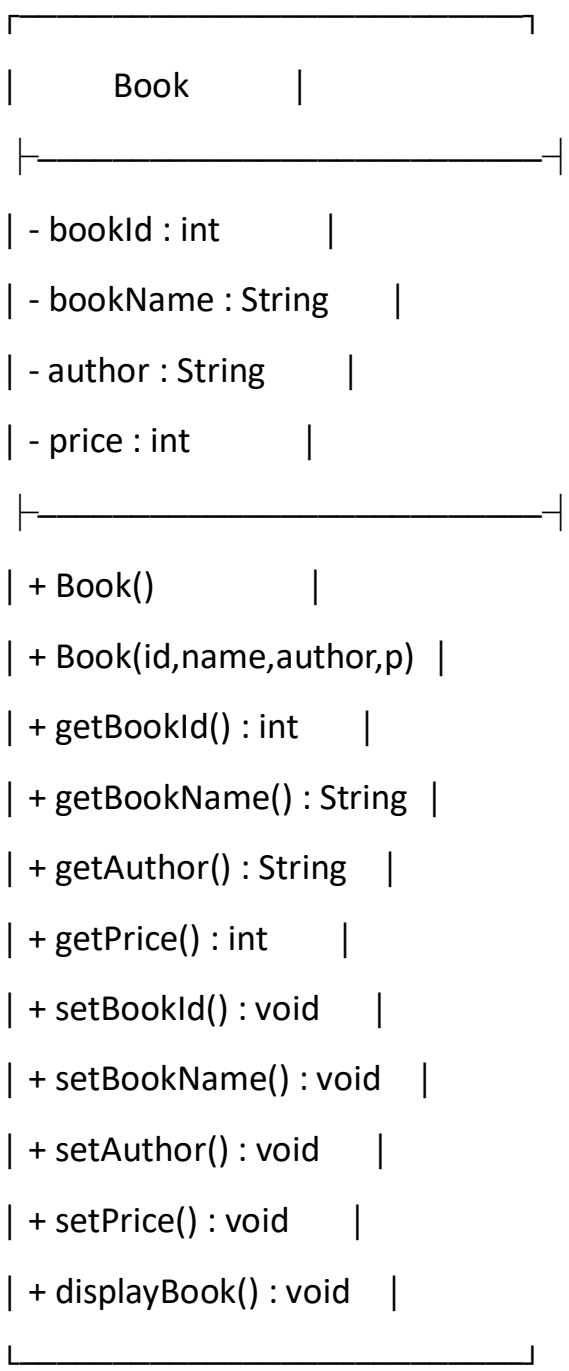
The output is:

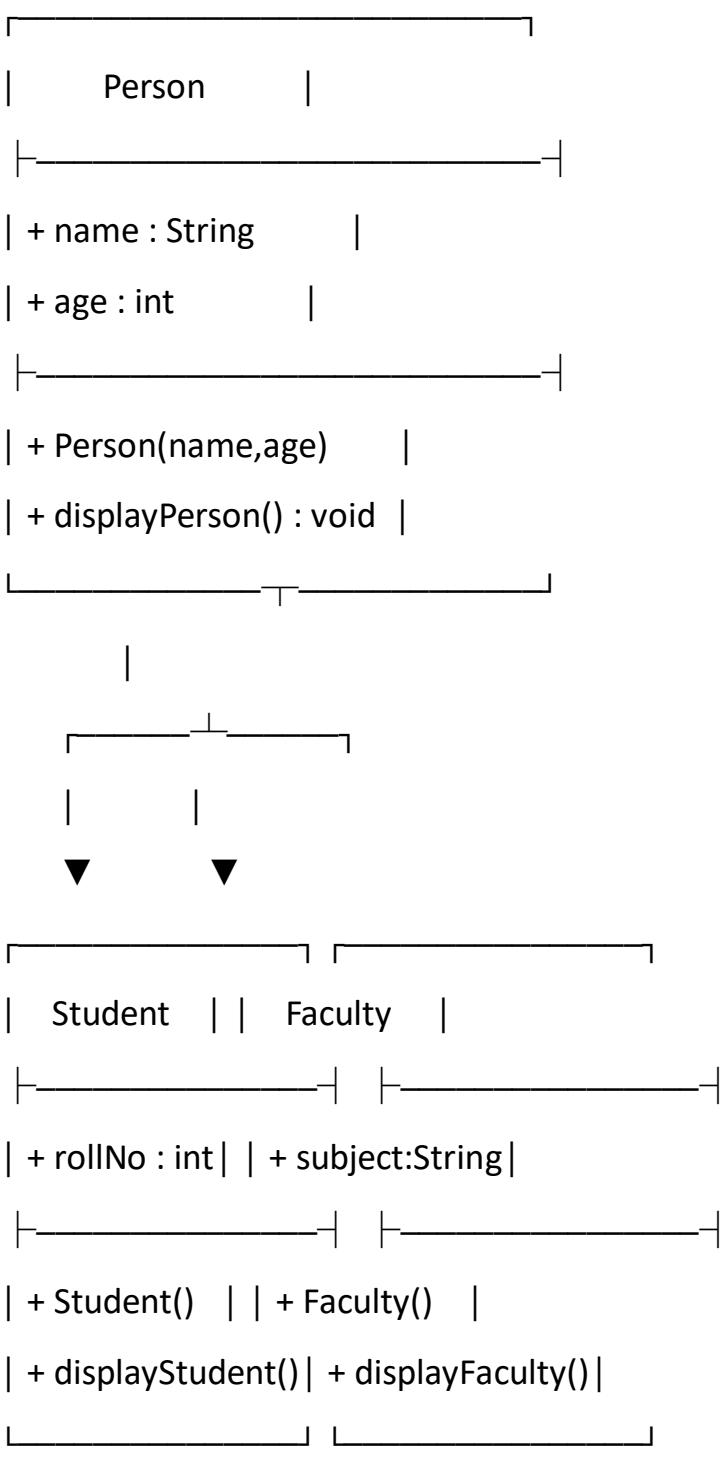
Printing Report

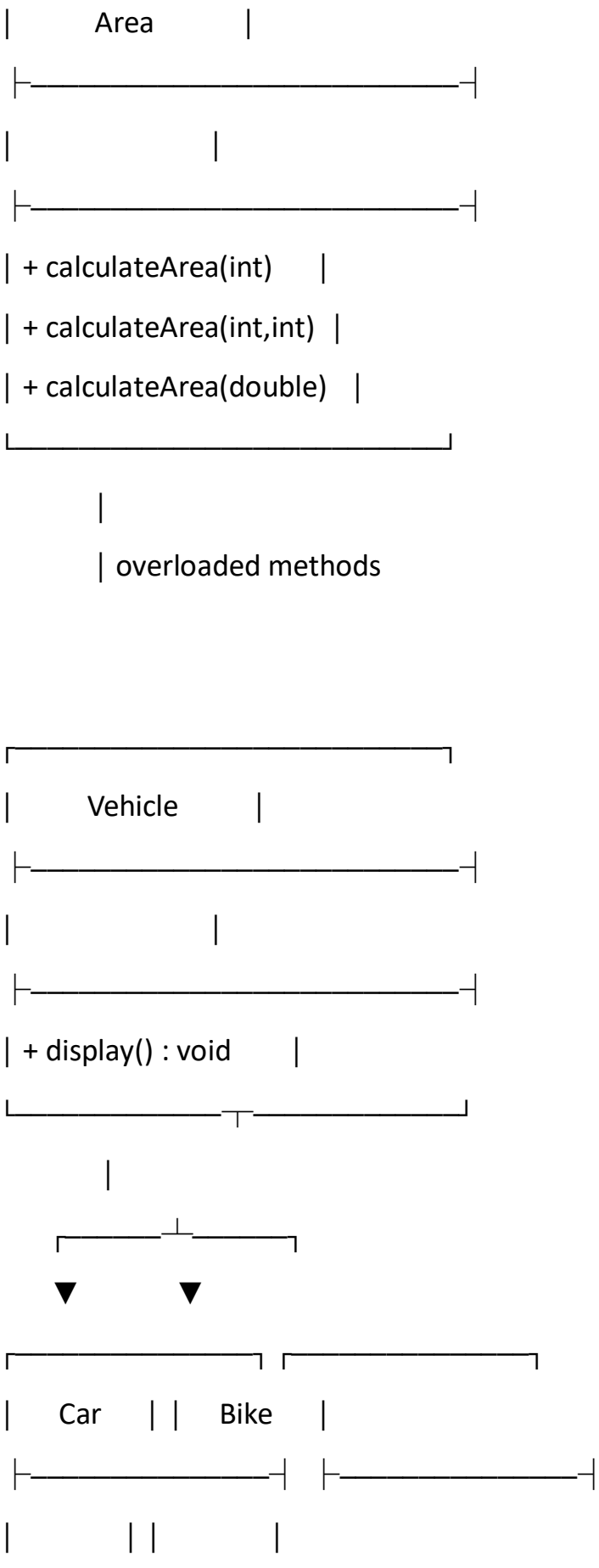
STEP 40:

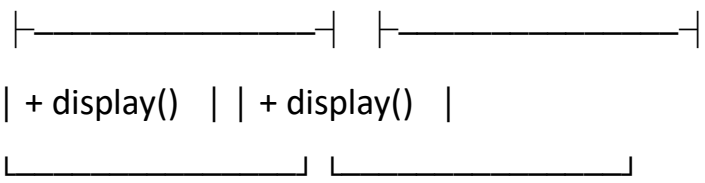
The program ends.

UML Class Diagram:

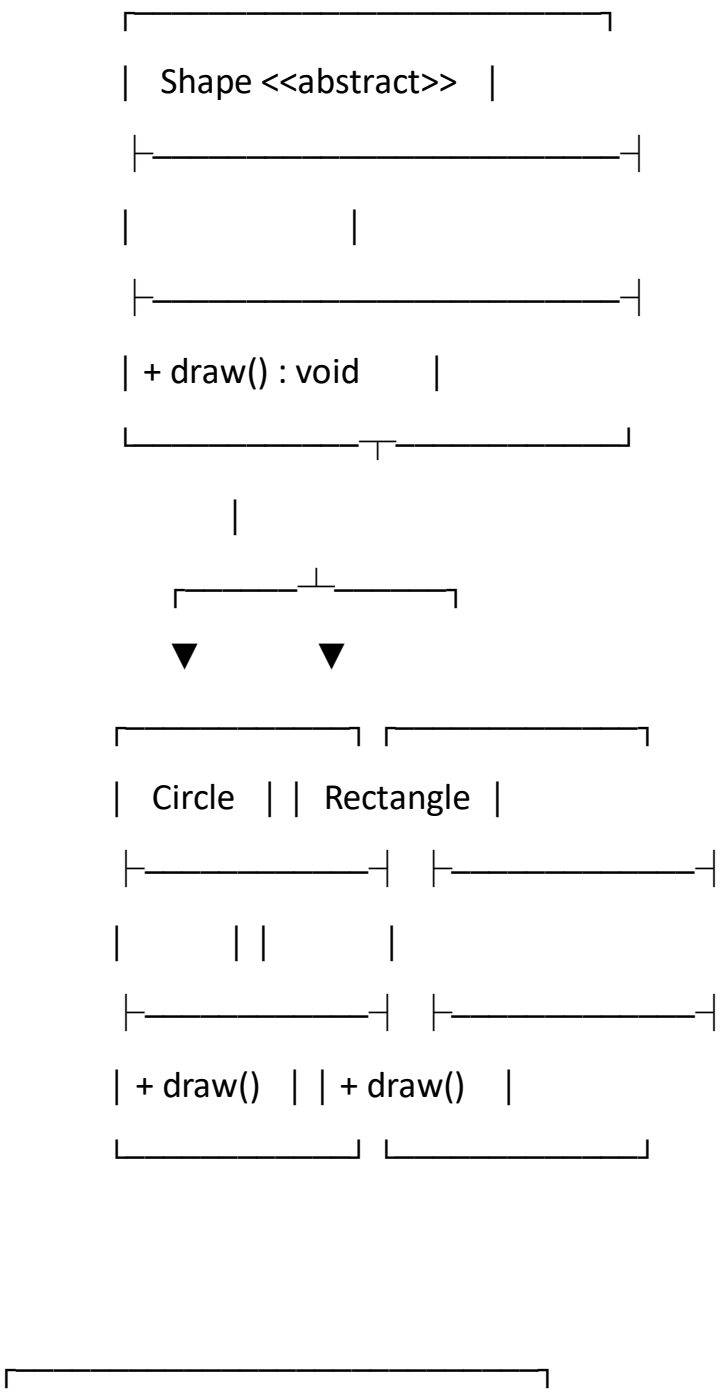


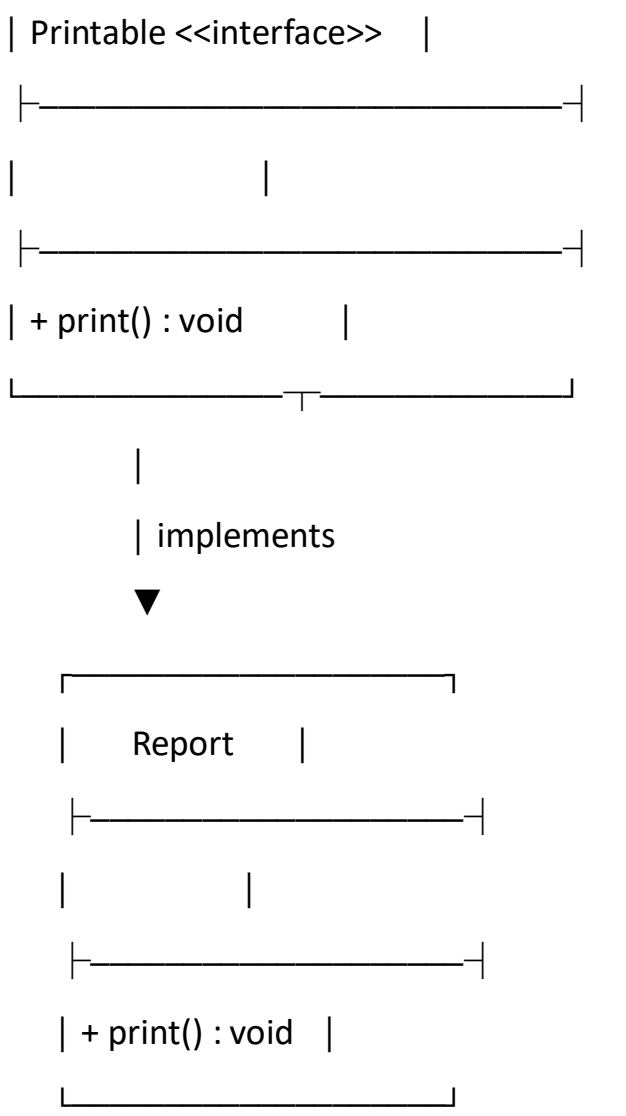

Person
+ name : String
+ age : int
+ Person(name,age)
+ displayPerson() : void
Student
+ rollNo : int
+ Student()
+ displayStudent()
Faculty
+ subject:String
+ Faculty()
+ displayFaculty()


Area
+ calculateArea(int)
+ calculateArea(int,int)
+ calculateArea(double)
overloaded methods
Vehicle
+ display() : void
Car
Bike



Method Overriding




Printable <<interface>>
+ print() : void
implements
Report
+ print() : void
LibMangSys
+ main(args:String[]) : void

Code:

```
package javacore;

// Encapsulation
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private int price;

    // Default constructor
    Book() {
        bookId = 123;
        bookName = "Malgudi Days";
        author = "RkNarayan";
        price = 299;
    }

    // Parameterized constructor
    Book(int bookId, String bookName, String author, int price) {
        this.bookId = bookId;
        this.bookName = bookName;
        this.author = author;
        this.price = price;
    }

    // Getters
```

```
int getBookId() {
   return bookId;
}

String getBookName() {
   return bookName;
}

String getAuthor() {
   return author;
}

int getPrice() {
   return price;
}

// Setters
void setBookId(int bookId) {
   this.bookId = bookId;
}

void setBookName(String bookName) {
   this.bookName = bookName;
}

void setAuthor(String author) {
```

```
        this.author = author;
    }

    void setPrice(int price) {
        this.price = price;
    }

    void displayBook() {
        System.out.println("Book Id: " + bookId);
        System.out.println("Book Name: " + bookName);
        System.out.println("Author: " + author);
        System.out.println("Price: " + price);
    }
}

// Parent class
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayPerson() {
```

```
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Inheritance
class Student extends Person {
    int rollNo;

    Student(String name, int age, int rollNo) {
        super(name, age);
        this.rollNo = rollNo;
    }

    void displayStudent() {
        displayPerson();
        System.out.println("Roll no: " + rollNo);
    }
}

// Inheritance
class Faculty extends Person {
    String subject;

    Faculty(String name, int age, String subject) {
        super(name, age);
```

```
        this.subject = subject;
    }

    void displayFaculty() {
        displayPerson();
        System.out.println("Sub: " + subject);
    }
}

// Method Overloading
class Area {

    // Square
    int calculateArea(int side) {
        return side * side;
    }

    // Rectangle
    int calculateArea(int length, int breadth) {
        return length * breadth;
    }

    // Circle
    double calculateArea(double radius) {
        return 3.14 * radius * radius;
    }
```

```
}

// Parent class
class Vehicle {
    void display() {
        System.out.println("This is a vehicle");
    }
}

// Method Overriding
class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a car");
    }
}

// Method Overriding
class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a bike");
    }
}

// Abstraction
```

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Draw circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Draw rectangle");
    }
}

// Interface
interface Printable {
    void print();
}

class Report implements Printable {
    @Override
    public void print() {
```

```
        System.out.println("Printing Report");
    }
}

// Main class
public class LibMangSys {

    public static void main(String[] args) {

        // Constructors and Encapsulation
        System.out.println("Book Details");

        Book b1 = new Book();
        Book b2 = new Book(108, "Life", "Thomas", 500);

        b1.displayBook();

        System.out.println();

        b2.displayBook();

        // Inheritance
        System.out.println("Inheritance");

        Student s = new Student("Ram", 20, 18);
        s.displayStudent();
```

```
System.out.println();

Faculty f = new Faculty("Ramesh", 38, "Maths");
f.displayFaculty();

// Method Overloading
System.out.println("Overloading");

Area a = new Area();

System.out.println("Area Square: " + a.calculateArea(4));
System.out.println("Area Rectangle: " + a.calculateArea(5, 6));
System.out.println("Area Circle: " + a.calculateArea(9.0));

// Method Overriding
System.out.println("Overriding");

Vehicle v1 = new Car();
Vehicle v2 = new Bike();

v1.display();
v2.display();

// Abstraction
System.out.println("Abstraction");
```

```
        Shape c = new Circle();
        Shape r = new Rectangle();

        c.draw();
        r.draw();

        // Interface
        System.out.println("Interface");

        Report re = new Report();
        re.print();
    }
}
```

```
Book Details
Book Id: 123
Book Name: Malgudi Days
Author: RkNarayan
Price: 299

Book Id: 108
Book Name: Life
Author: Thomas
Price: 500
Inheritance
Name: Ram
```

Age: 20

Roll no: 18

Name: Ramesh

Age: 38

Sub: Maths

Overloading

Area Square: 16

Area Rectangle: 30

Area Circle: 254.34

Overriding

This is a car

This is a bike

Abstraction

Draw circle

Draw rectangle

Interface

Printing Report